

# java\_wrapper

## logger module-attribute

```
logger = getLogger(__name__)
```

## JavaWrapper

```
JavaWrapper(pyjnius_java)
```

A wrapper for jnius generated objects.

The jnius framework already does a substantial work when converting java objects into python object. Unfortunately, this work is more focused on java native types. This class complements this functionally by providing additional transformation functions that can automatically convert objects from both the Java native library as well as some types from the Feedzai common library. As such, this class should always be used to wrap pyjnius generated objects.

An example of the usage for this class would be: Given a java class Car, which contains a Car#getWheels which returns a java.util.List. A JavaWrapper for an instance of this class would return a python array of Wheel objects (the conversion from java.util.List to a python array would occur automatically). Note that pyjnius only converts java.util.List objects into a pyjnius generated class for java.util.List, thus making the code that interacts with it more verbose.

Note also that the conversions are transitive. A java method that returns a List< is automatically converted into a python array of arrays of strings.

Another useful feature of this class is that it is able to log details for the JVM exceptions thrown from the pyjnius JVM. The details include the java stacktrace, which should be useful to debug java code that pyjnius interacts with.

Class variables "java\_classref" and "java\_class" are to be defined by every subclass, which refer to the java class that the python class can represent. Imagining for a DataSource, it basically says, if you give me a "com.feedzai.common.dsapi.DataSource" I can convert it to python. They will be automatically added to a map available to every class, so the return types coming from java can accurately/automatically be converted to their Python friends. In this case, if no good match is found, it defaults to this JavaWrapper class.

Finally, note that if you intend to change the init's signature of your new subclass, be a pal and define a static method named "auto\_cast" that must have one parameter: the java object that represents our python counterpart. That way, our auto-cast system will cleverly pick up that method and work smoothly. This avoids extra ".unwrap()" and ".get()" that make the use of the api by the end-user less intuitive.

There are examples of this auto-cast method in the models package and types module.

Parameters:

| Name         | Type | Description                  | Default  |
|--------------|------|------------------------------|----------|
| pyjnius_java |      | The original pyjnius object. | required |

## subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```